

TRUTOUCH

RONIT RAJIB PATTNAIK

PHASE: 1

START DATE: June 2 2025

Contents

Introduction 1
1.1 Project Background 1
1.2 Project Objectives 1
2. Materials and Tools 3
2.3 Software Tools 4
3. Methodology 5
3.1 Version History & Evolution 5
3.2 Circuit Design 6
3.3 Hand Construction 7
3.4 Software Implementation 8
4. Challenges and Solutions 10
4.2. Software Challenges 11
APPENDIX A: CODES 12
PYTHON CODE 12
ARDUINO CODE 15
APPENDIX B: ALL TRIAL MODELS 18

Introduction

1.1 Project Background

Inspiration and Motivation

The proposed work aims to develop a **low-cost, EMG-based prosthetic hand** designed specifically for underserved users working in the **tertiary workforce**—including bus conductors, barbers, sanitation workers, delivery personnel, and others whose livelihood depends on basic hand functions but who cannot afford high-end prosthetic solutions. Current advanced bionic hands offer multi-degree-of-freedom control and highly human-like dexterity, yet remain financially inaccessible to a large portion of the population, particularly in developing countries. Conversely, the low-cost prosthetic options available today are mostly cosmetic and provide little or no functional movement, creating a significant usability gap.

Our vision is to build a **functional, reliable, and economically accessible prosthetic hand** that focuses on the most essential grasping tasks: **opening and closing the hand** for holding, supporting, and performing basic daily activities. Instead of expensive multi-electrode EMG systems, the design leverages **single-channel EMG (and optionally ECG) signals**, enhanced through **AI-driven filtering and pattern-recognition algorithms** to reliably extract intent even from noisy biological inputs. This allows the device to maintain extremely low hardware cost while still achieving practical performance.

1.2 Project Objectives

Primary Objectives

1. **Develop a low-cost EMG-controlled prosthetic hand** capable of reliable open–close functionality using a single-channel biosignal input.
2. **Implement AI-based filtering and intent-detection algorithms** to compensate for noisy, low-quality bioelectrical signals without requiring expensive sensors.
3. **Design a robust, lightweight mechanical hand** that prioritizes essential grasping tasks suitable for everyday work environments.
4. **Ensure the system remains affordable, maintainable, and easy to use**, enabling adoption by individuals in the tertiary workforce.

Long-Term Vision

1. **Expand from basic grasping to multi-finger voluntary control**, incorporating advanced machine-learning models and multiple signal features.
2. **Develop a fully integrated prosthetic arm ecosystem**, including onboard battery management, waterproofing, modular components, and ergonomic refinement.
3. **Scale manufacturing for widespread accessibility**, positioning the device as a globally competitive low-cost alternative produced from India.
4. **Create a smart adaptive prosthetic platform** capable of learning user-specific motion patterns over time, improving accuracy, comfort, and natural movement.

2. Materials and Tools

2.1. Hardware Components

- **Arduino Uno** – Acts as the central microcontroller that receives commands from the computer and controls the servo motors.
- **SG90 Servo Motors (x5)** – Miniature servo motors used to actuate each finger individually. Operates on PWM (Pulse Width Modulation) signals from Arduino.
- **PCA9685 Servo Driver** – A 16-channel PWM driver that offloads PWM generation from Arduino, ensuring smooth multi-servo operation.
- **Elastic Rubber Bands** – Connected to each 3D printed finger to serve as return springs, restoring fingers to their open position after curling.
- **Nylon Fishing Wire (High-Strength)** – Acts as tendons, routed from servo horns to finger tips, enabling curling motion when the servo pulls the wire.
- **Wires and Jumper Cables** – Used for connecting the Arduino, servos, and external power to ensure a stable and modular setup.
- **Breadboard and Connectors** – Temporary circuit building platform for testing connections and power management before permanent wiring.
- **5V 5A SMPS (Switched-Mode Power Supply)** – Dedicated power source to handle the current demands of multiple servos.
- **(Reserved) EMG Sensor** – Planned for future integration for biological signal-based control, though not used in the current prototype.

2.2 Mechanical Materials

The prosthetic hand structure was developed in two stages — an initial cardboard prototype for proof-of-concept, followed by a 3D-printed PLA model for improved durability, precision, and functionality.

Prototype Stage (Cardboard Version):

- Cardboard: Used for constructing the base of the hand and palm housing, due to its availability and ease of shaping.
- Rubber Bands: Served as tendon return mechanisms to restore finger position after flexion.
- Thread/String: Used common white household sewing thread (substitute for fishing line) to connect the servo arms to the cardboard fingers for curling motion.
- Glue: Fevikwik used for bonding components and reinforcing finger joints.
- Toothpicks: Used to connect finger segments and allow pivot movement.

Upgraded Stage (3D-Printed PLA Version):

- PLA Filament: Used to fabricate a durable, lightweight, and precise prosthetic hand structure.
- Elastic Rubber Bands: Act as tendon return systems for the 3D-printed fingers, providing passive extension.
- Nylon Fishing Wire: Connected between the servo horns and finger joints to create curling motion when pulled.
- Screws and Hinges: Used to assemble 3D-printed finger segments and enhance joint stability.
- 3D Printer: Used to manufacture the custom SolidWorks-designed prosthetic hand parts.

2.3 Software Tools

The control system, signal processing, and mechanical design were developed using a combination of programming environments and CAD tools:

- **Arduino IDE** – Used to write and upload code to the Arduino board, handling servo control logic and integration with the PCA9685 servo driver.
- **Python (3.10) with OpenCV and MediaPipe** – Python scripts handled gesture detection using MediaPipe's hand tracking solution. OpenCV was used for real-time webcam input processing.
- **PySerial (Python ↔ Arduino)** – Enabled serial communication between Python and Arduino to send gesture-based commands to the servo control system.
- **TinkerCAD** – Used to create circuit diagrams and system schematics for documentation, testing, and troubleshooting.
- **SolidWorks** – Fully utilized for designing a custom 3D model of the prosthetic hand, including finger joints and mechanical linkages, replacing earlier reliance on pre-made models.
- **Cura**– Used to convert SolidWorks designs into G-code for 3D printing.

3. Methodology

3.1 Version History & Evolution

The development of the project followed a phased approach, evolving through multiple prototypes. While the long-term aim is to integrate EMG-controlled, high-functionality finger movement, the initial stages focused on validating the mechanical structure and control logic. This allowed the system to be tested and refined before integrating complex biological inputs.

- **Version 1: Basic Servo Test**
    - **Goal:** Confirm servo motor functionality.
    - **Approach:** Connected a single SG90 servo to the Arduino and controlled its position using a basic sketch.
    - **Outcome:** Successful; verified power requirements and basic control logic.
- **Version 2: Single-Finger Control via Python**
    - **Goal:** Enable servo control from PC using Python.
    - **Approach:** Sent serial commands from Python to Arduino to control one servo.
    - **Outcome:** Functional but communication stability was occasionally inconsistent.
- **Version 3: Gesture Recognition Integration**
    - **Goal:** Implement basic gesture detection using a webcam.
    - **Approach:** Used MediaPipe to detect open/closed hand gestures and send corresponding commands to Arduino.
    - **Outcome:** Successfully triggered servo movement based on gestures; demonstrated proof-of-concept for vision-based control.
- **Version 4: Full Hand Control (5 Fingers)**
    - **Goal:** Achieve individual control of five servos corresponding to each finger.
    - **Approach:** Integrated PCA9685 driver for smoother, synchronized multi-servo operation.
    - **Outcome:** Achieved realistic, coordinated finger movements with stable performance.

- **Version 5: Cardboard Mechanical Prototype**
    - **Goal:** Create a low-cost physical model to validate tendon-driven motion.
    - **Approach:** Constructed the hand from cardboard with thread tendons and rubber band return mechanisms.
    - **Outcome:** Successfully simulated prosthetic hand motion; provided valuable data for tendon placement and servo mounting.
- **Final Prototype: 3D-Printed PLA Model**
    - **Goal:** Upgrade from cardboard to a durable, custom-designed 3D-printed prosthetic hand.
    - **Approach:** Designed the entire hand in SolidWorks, printed parts in PLA, used nylon fishing wire for tendon actuation and elastic bands for passive extension.
    - **Outcome:** Achieved a mechanically robust, realistic hand structure ready for EMG sensor integration and AI-driven functional modes.

3.2 Circuit Design

The electronic control system centers around an Arduino Uno, a PCA9685 servo driver, and multiple SG90 servo motors. Power is supplied using a 5V 5A SMPS for high-current servo operation, with careful grounding to ensure stable performance.

1. Power Handling

- **SMPS (5V, 5A)** powers all servo motors through the PCA9685's **V+** terminal.
- PCA9685 **V+ pin** is connected directly to the SMPS positive output for high-current delivery.
- Arduino is powered via USB from the PC during development, or through an external barrel jack if standalone.
- This configuration ensures the Arduino logic and servo motors operate independently, preventing brownouts.

2. Ground Logic

- A **common ground** is shared between:
    - SMPS ground (GND)
    - PCA9685 GND
    - Arduino GND

- Common grounding ensures the PWM signal levels from Arduino to PCA9685 remain consistent, preventing erratic servo movement.

3. Control Signal Path

- PCA9685 connects to Arduino via **I2C** (SCL, SDA pins).
- Each servo's **signal (yellow/orange) wire** connects to one of the PCA9685's 16 output channels.
- The **tendon-driven mechanism** uses:
    - **Nylon fishing wire** (servo horn → finger joint) for curling motion.
    - **Elastic rubber bands** (finger joint → base) for automatic return to the open position.
- Arduino communicates with the PCA9685 using the **Adafruit PWM Servo Library** for precise and smooth servo control.

3.3 Hand Construction

The prosthetic hand was manually constructed using cardboard for the palm and base, later transitioning to custom 3D-printed components for improved precision and durability.

1. Mechanism

- Nylon fishing wire attached from each fingertip to a servo horn acted as tendons, enabling curling motion.
- Elastic rubber bands at the back of joints provided passive return force, simulating extensor muscles.

2. Assembly Stages – Cardboard Version

1. Palm structure assembled from layered cardboard.
2. Finger skeletons cut and joined using glue and toothpicks; small counterweights added to fingertips to ensure natural opening when extended.
3. Tendon threads routed through guides and connected to servo horns.
4. Servos mounted on the palm base.
5. Motion tested and adjusted for smoothness.

3. Model Evolution

- Version A – Cardboard Prototype: Served as a low-cost platform for tendon routing validation and motion proof-of-concept.

- Version B – Basic 3D-Printed Model: Custom-designed in SolidWorks, with precise finger joint geometry and improved servo mounts.



Figure 1: Cardboard Model

3.4 Software Implementation

The control system integrates Python (for real-time gesture recognition) with Arduino (for servo motor actuation). The entire flow is based on detecting hand gestures, converting them into binary finger states, and triggering appropriate servo positions via serial communication.

1. MediaPipe + OpenCV for Gesture Detection

- OpenCV captures live video frames from the webcam.
- MediaPipe Hands processes each frame to identify 21 key hand landmarks.
- A custom logic checks the relative position of each fingertip to its corresponding lower joint (e.g., tip vs. MCP joint).
- If a fingertip is above the joint (in camera view), the finger is considered extended (open = 1); otherwise, folded (closed = 0).
- This results in a 5-bit binary array like [1, 1, 0, 0, 0], representing the thumb to pinky fingers.
- This binary list is sent as a string via serial — for example: "11000\n".

2. Python-to-Arduino Communication

- Communication is established using pyserial at 9600 baud rate.
- Python sends a 5-character binary string, followed by a newline (\n), representing the desired finger states.
- Each bit in the string corresponds to one servo (e.g., bit 0 → thumb servo, bit 1 → index finger, etc.).

- Example:
    - "11111\n" → all fingers open
    - "00000\n" → all fingers closed
    - "10001\n" → thumb and pinky open; others closed

3. Arduino Servo Control

- The Arduino reads the 5-bit binary string from the serial buffer.
- Each bit is mapped to a specific servo motor.
- For each servo:
    - If the bit is "1" → move servo to 180° (open position)
    - If the bit is "0" → move servo to 0° (closed position)

4. Challenges and Solutions

4.1. Hardware challenges

- **Component Availability & Selection**
  One of the primary challenges was sourcing the correct components, especially given the budget and local availability. While components like servos and Arduino were relatively easy to find, finding a stable power supply capable of powering all five servos proved difficult. Most options failed to deliver consistent current.
  Solution: After testing multiple options, a 5V 5A SMPS (Switched-Mode Power Supply) was identified and used. It delivered sufficient and stable current to run all servos simultaneously without overheating or voltage drops.

- **Servo Jitter and Inconsistent Movement**
  Initially, the servos would jitter, stutter, or behave unpredictably, especially when multiple motors were active.
  Cause: This was traced back to insufficient or unstable power, as well as lack of a proper shared ground.
  Solution: Once the SMPS was integrated and all components shared a common ground, the jittering stopped and servo performance became smooth and consistent.

- **Size Inconsistency in 3D Printed Components** *(3D Model Specific)*
  During the first iterations of the 3D-printed hand, component sizes did not match expectations. Joints were either too loose or too tight, and servo slots lacked proper fit. This was partly due to limited prior experience with SolidWorks dimensioning and tolerances.
  Solution: Measured all physical components with calipers before redesigning parts in SolidWorks. Applied tolerance adjustments (e.g., ±0.2 mm) to ensure proper fits after printing. Multiple reprints were made until dimensions were correct.

- **Hinge Arrangement Difficulties** *(3D Model Specific)*
  Early hinge designs either restricted movement or introduced excessive friction. Printed hinges also wore down quickly.
  Solution: Shifted to a pin-based hinge system using metal pins/toothpicks, which provided smoother rotation and durability. Redesigned the hinge geometry in SolidWorks to match the tendon routing path.

4.2. Software Challenges

- **Hand Detection Confusion (Left/Right Orientation)**
  MediaPipe often confused the left and right hand, especially when mirrored video feed was used. This caused unexpected behavior, particularly in interpreting thumb movement, which varies depending on hand orientation.
  Solution: The logic was adjusted to detect and lock the correct handedness during initialization. Thumb detection rules were rewritten to adapt dynamically to left/right context.

- **Incorrect Binary Command Formation**
  At one point, the Python script was sending malformed binary strings, which led the Arduino to interpret the data incorrectly. This often resulted in all servos returning to 0° (closed position) unintentionally.
  Solution: The code was restructured to ensure:
    - Binary strings were always exactly 5 bits (e.g., "10100\n").
    - Commands were properly terminated with a newline character (\n).

- **SolidWorks File Handling Issues** *(3D Model Specific)*
  While designing the 3D model in SolidWorks, the large number of part and assembly files open simultaneously caused Windows Explorer to crash and SolidWorks to become unstable.
  Solution: Organized all files into structured project folders, closed unused files during editing, and regularly cleared SolidWorks' temporary cache. Also adopted incremental saving to prevent data loss.

APPENDIX A: CODES

PYTHON CODE

```
import cv2
import mediapipe as mp

import serial
import time

# === CONFIG ===
SERIAL_PORT = 'COM8'  # Change if needed
BAUD_RATE = 9600

# === SERIAL SETUP ===
try:
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE)
    print(f"☑ Connected to {SERIAL_PORT}")
    time.sleep(2)
except Exception as e:
    print(f"❌ Serial Error: {e}")
    exit()

# === MEDIAPIPE SETUP ===
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(static_image_mode=False,
                       max_num_hands=1,
                       min_detection_confidence=0.7,
                       min_tracking_confidence=0.7)
mp_draw = mp.solutions.drawing_utils
tip_ids = [4, 8, 12, 16, 20]

cap = cv2.VideoCapture(0)

# Track last state for each finger
last_finger_states = [None, None, None, None, None]  # [thumb, index, middle,
ring, pinky]
last_command_time = 0

# === FUNCTION TO GET FINGER BINARY ===
def get_finger_binary(lm_list, label):
    fingers = []

    # Thumb
    if label == "Right":
        if lm_list[4][1] < lm_list[3][1]:
            fingers.append(1)
        else:
```

```
            fingers.append(0)
    else:  # Left hand
        if lm_list[4][1] > lm_list[3][1]:
            fingers.append(1)
        else:
            fingers.append(0)

    # Index to pinky
    for i in range(1, 5):
        if lm_list[tip_ids[i]][2] < lm_list[tip_ids[i] - 2][2]:
            fingers.append(1)
        else:
            fingers.append(0)

    return fingers

# === MAIN LOOP ===
while True:
    success, frame = cap.read()
    if not success:
        continue

    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(rgb)

    if results.multi_hand_landmarks:
        for idx, hand_landmarks in enumerate(results.multi_hand_landmarks):
            hand_label = results.multi_handedness[idx].classification[0].label

            lm_list = []
            h, w, _ = frame.shape

            for id, lm in enumerate(hand_landmarks.landmark):
                cx, cy = int(lm.x * w), int(lm.y * h)
                lm_list.append((id, cx, cy))

            if lm_list:
                fingers = get_finger_binary(lm_list, hand_label)
                current_binary = ''.join(str(bit) for bit in fingers)

                # Individual finger states
                thumb_state = fingers[0]    # Thumb
                index_state = fingers[1]    # Index
                middle_state = fingers[2]   # Middle
                ring_state = fingers[3]     # Ring
                pinky_state = fingers[4]    # Pinky
```

```
                current_states = [thumb_state, index_state, middle_state,
ring_state, pinky_state]
                finger_names = ["Thumb", "Index", "Middle", "Ring", "Pinky"]

                print(f"🤚 {hand_label} Hand - T:{thumb_state}
I:{index_state} M:{middle_state} R:{ring_state} P:{pinky_state} | Full:
{current_binary}")

                # Check if any finger state changed (with timing control)
                current_time = time.time()
                if current_time - last_command_time > 0.1:  # 100ms delay

                    state_changed = False
                    for i in range(5):
                        if current_states[i] != last_finger_states[i]:
                            state_changed = True
                            break

                    if state_changed:
                        # Calculate angles for all fingers
                        thumb_angle = 180 if thumb_state == 1 else 0
                        index_angle = 180 if index_state == 1 else 0
                        middle_angle = 180 if middle_state == 1 else 0
                        ring_angle = 180 if ring_state == 1 else 0
                        pinky_angle = 180 if pinky_state == 1 else 0

                        # Send command in format
"T:angle,I:angle,M:angle,R:angle,P:angle"
                        command =
f"T:{thumb_angle},I:{index_angle},M:{middle_angle},R:{ring_angle},P:{pinky_ang
le}\n"
                        ser.write(command.encode())
                        ser.flush()

                        print(f"☑ Sent: T={thumb_angle}° I={index_angle}°
M={middle_angle}° R={ring_angle}° P={pinky_angle}°")

                        # Update last states
                        last_finger_states = current_states.copy()
                        last_command_time = current_time

            mp_draw.draw_landmarks(frame, hand_landmarks,
mp_hands.HAND_CONNECTIONS)

    # Display current finger states on screen
    if 'fingers' in locals():
        # Individual finger status
```

```
        cv2.putText(frame, f"T:{fingers[0]} I:{fingers[1]} M:{fingers[2]}
R:{fingers[3]} P:{fingers[4]}",
                    (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

        # Full binary
        cv2.putText(frame, f"Binary: {''.join(str(bit) for bit in fingers)}",
                    (10, 70), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 0), 2)

        # Pin assignments
        cv2.putText(frame, "Pins: T=9, I=10, M=11, R=12, P=13",
                    (10, 110), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 255),
2)

    cv2.imshow("5-Finger Servo Control", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
ser.close()
cv2.destroyAllWindows()
```

ARDUINO CODE

```
#include <Wire.h>

#include <Adafruit_PWMServoDriver.h>

// === PCA9685 SETUP ===
Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver(0x40);  // Default I2C
address

// Servo pulse length constants for PCA9685
#define SERVOMIN  102  // 0.5ms pulse (0 degrees)
#define SERVOMAX  512  // 2.5ms pulse (180 degrees)
#define SERVO_FREQ 50  // Analog servos run at ~50 Hz updates

// PCA9685 Channel assignments (instead of pins)
#define THUMB_CHANNEL   0   // Channel 0 - Thumb
#define INDEX_CHANNEL   1   // Channel 1 - Index finger
#define MIDDLE_CHANNEL  2   // Channel 2 - Middle finger
#define RING_CHANNEL    3   // Channel 3 - Ring finger
#define PINKY_CHANNEL   4   // Channel 4 - Pinky finger

int lastThumbAngle = -1;
int lastIndexAngle = -1;
int lastMiddleAngle = -1;
int lastRingAngle = -1;
int lastPinkyAngle = -1;
```

```
void setup() {
  Serial.begin(9600);

  // Initialize PCA9685
  pca.begin();
  pca.setOscillatorFrequency(27000000);  // Set to 27MHz internal oscillator
frequency
  pca.setPWMFreq(SERVO_FREQ);            // Analog servos run at ~50 Hz
updates

  delay(100);

  // Initialize all servos to 0 degrees using PCA9685
  moveServo(THUMB_CHANNEL, 0);
  moveServo(INDEX_CHANNEL, 0);
  moveServo(MIDDLE_CHANNEL, 0);
  moveServo(RING_CHANNEL, 0);
  moveServo(PINKY_CHANNEL, 0);

  Serial.println("Arduino Ready - 5 Servo Control (All Fingers) - PCA9685
Mode");
}

void loop() {
  if (Serial.available()) {
    // Read the complete line (expecting format like
"T:180,I:0,M:180,R:0,P:180")
    String command = Serial.readStringUntil('\n');
    command.trim();

    if (command.length() > 0) {
      // Parse all finger angles (same parsing logic as before)
      int thumbAngle = extractAngle(command, "T:");
      int indexAngle = extractAngle(command, "I:");
      int middleAngle = extractAngle(command, "M:");
      int ringAngle = extractAngle(command, "R:");
      int pinkyAngle = extractAngle(command, "P:");

      // Move each servo only if angle changed (using PCA9685 channels)
      moveServoIfChanged(THUMB_CHANNEL, thumbAngle, lastThumbAngle, "Thumb");
      moveServoIfChanged(INDEX_CHANNEL, indexAngle, lastIndexAngle, "Index");
      moveServoIfChanged(MIDDLE_CHANNEL, middleAngle, lastMiddleAngle,
"Middle");
      moveServoIfChanged(RING_CHANNEL, ringAngle, lastRingAngle, "Ring");
      moveServoIfChanged(PINKY_CHANNEL, pinkyAngle, lastPinkyAngle, "Pinky");
    }
```

```
    // Clear any remaining data in the buffer
    while (Serial.available()) {
      Serial.read();
    }
  }
}

// Function to extract angle from command string (UNCHANGED)
int extractAngle(String command, String prefix) {
  int startIndex = command.indexOf(prefix);
  if (startIndex == -1) return -1;

  int colonIndex = startIndex + prefix.length();
  int commaIndex = command.indexOf(",", colonIndex);

  if (commaIndex == -1) {
    // Last parameter, no comma
    return command.substring(colonIndex).toInt();
  } else {
    return command.substring(colonIndex, commaIndex).toInt();
  }
}

// Function to move servo only if angle changed (modified for PCA9685)
void moveServoIfChanged(int channel, int newAngle, int &lastAngle, String
fingerName) {
  if (newAngle != lastAngle && (newAngle == 0 || newAngle == 180)) {
    moveServo(channel, newAngle);
    lastAngle = newAngle;
    Serial.print(fingerName + " moved to: ");
    Serial.println(newAngle);
    delay(15);
  }
}

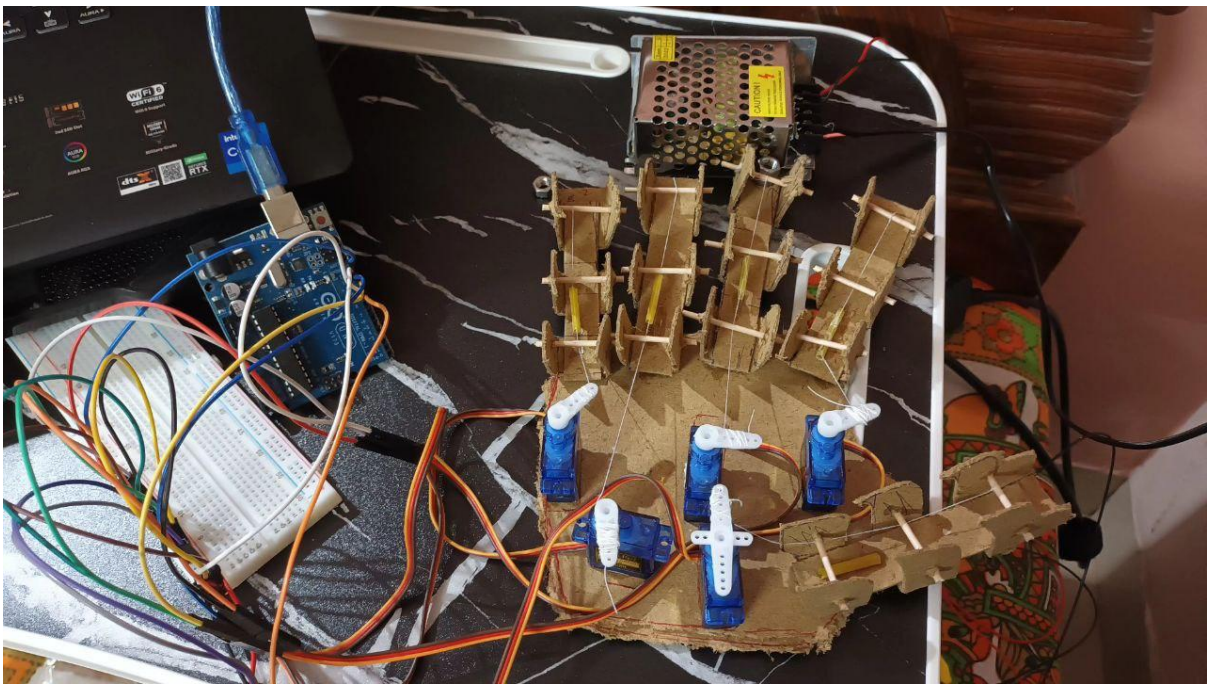
// New function to move servo using PCA9685
void moveServo(int channel, int angle) {
  // Constrain angle to valid range
  angle = constrain(angle, 0, 180);

  // Convert angle to pulse length
  int pulseLength = map(angle, 0, 180, SERVOMIN, SERVOMAX);

  // Set PWM on the specified channel
  pca.setPWM(channel, 0, pulseLength);
}
```

APPENDIX B: ALL TRIAL MODELS

Cardboard Model



3D Printed Model

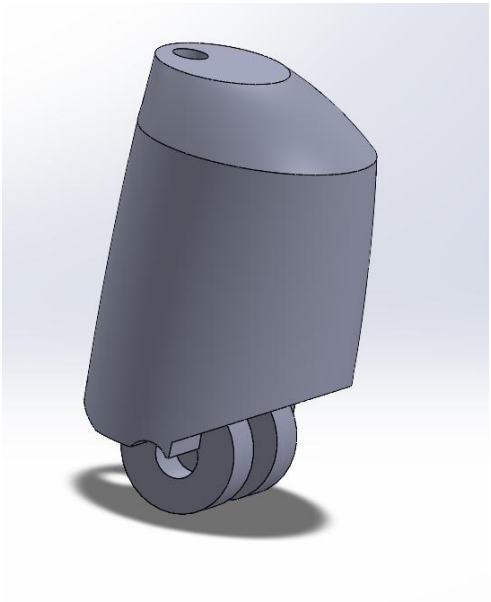


Figure 4: Concept V1: No Space for rope

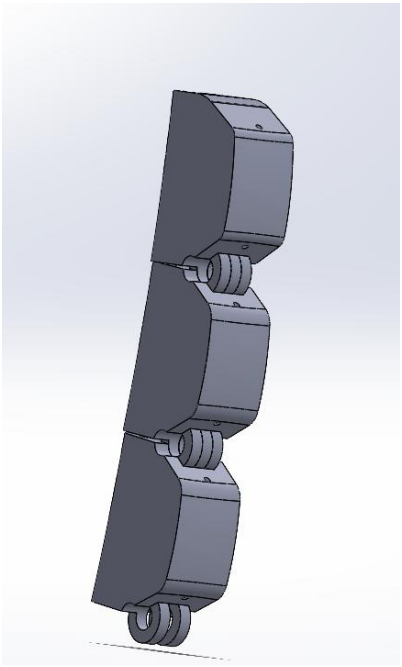


Figure 5: Concept V2: Weak hinge

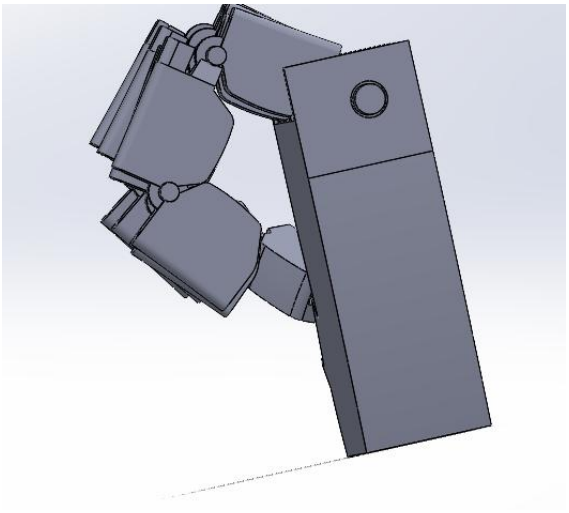


Figure 6: Concept V3: No grip size too big

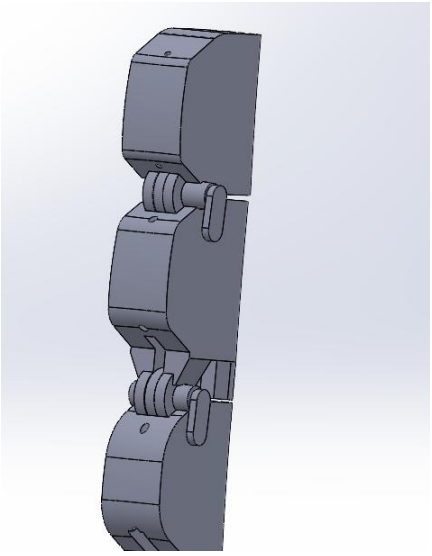


Figure 7: V3: No proper finger attachment system

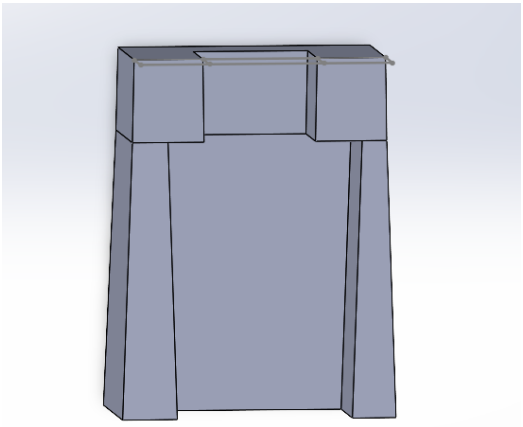


Figure 8: Forearm V1: no base or structure

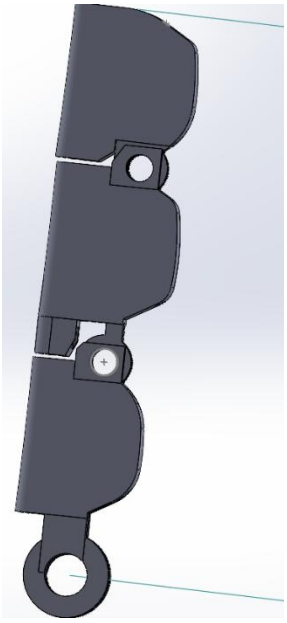


Figure 9: Finger V2: No proper finger attachment

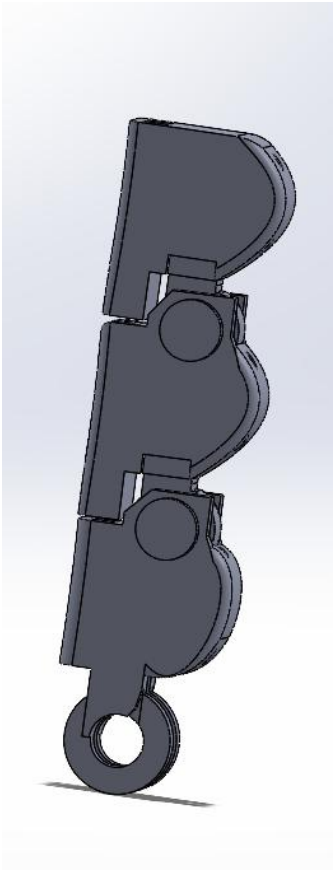


Figure 10: V4: Final Finger Version
Proper Attachment System
Proper Rope Space

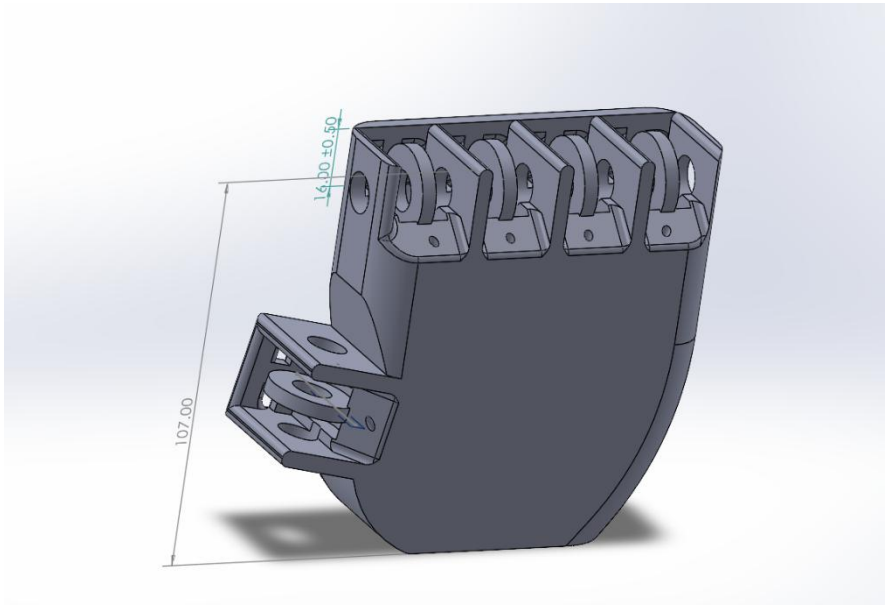


Figure 11: V2: Final Palm
Proper Finger to palm attachment
Proper chamfer for curling

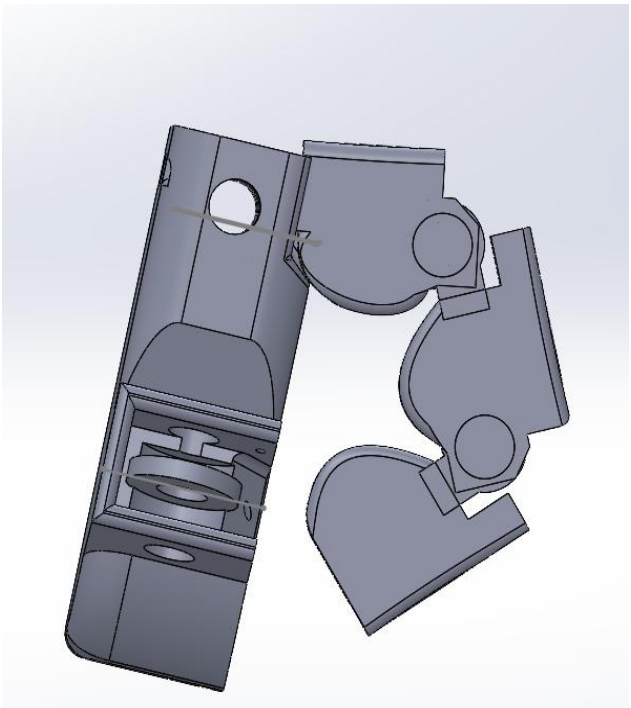


Figure 12: Proper with no interference

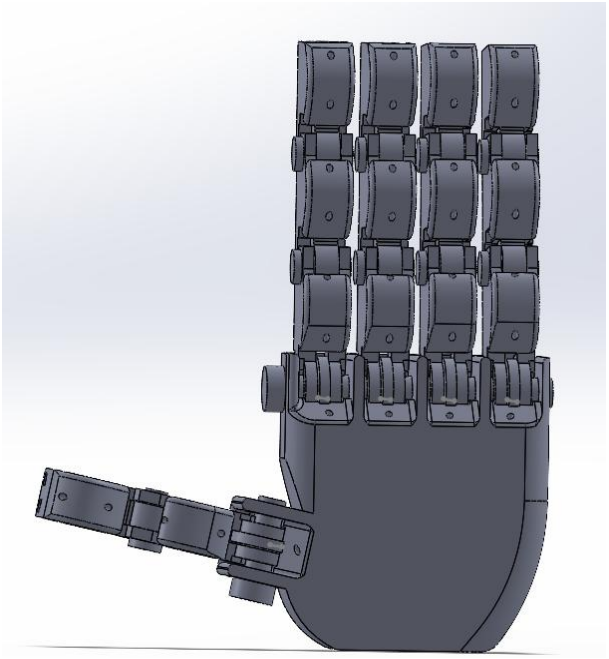


Figure 13: Final palm
Proper Length and Curl

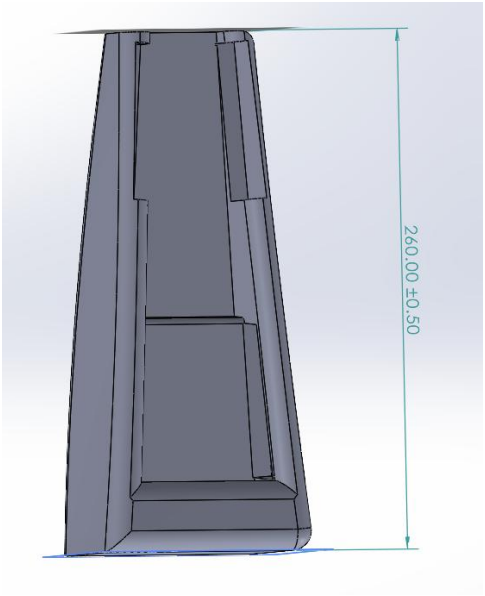


Figure 14: V2: Forearm
Proper Structure
Proper base
Enough Space for Servo

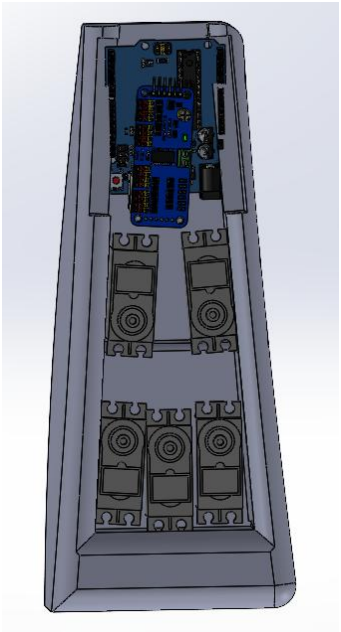


Figure 15: Assembly with Servo and micro-controllers

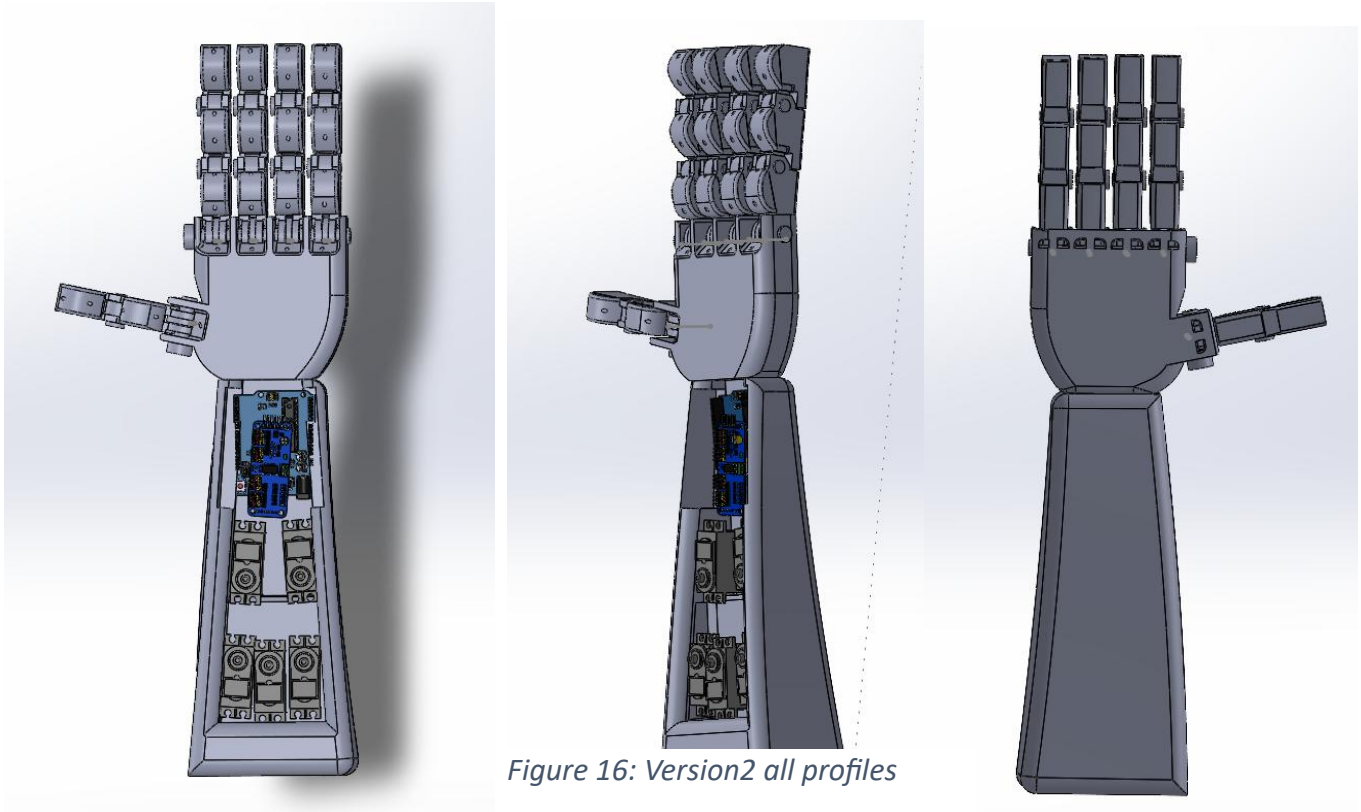


Figure 16: Version2 all profiles

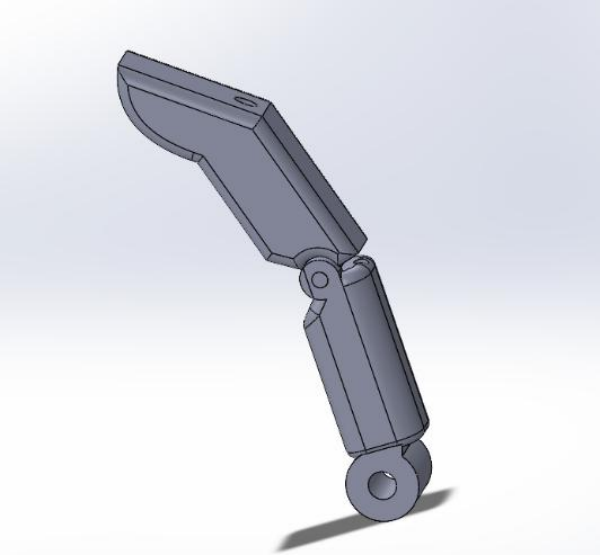


Figure 17: V2.5 finger concept

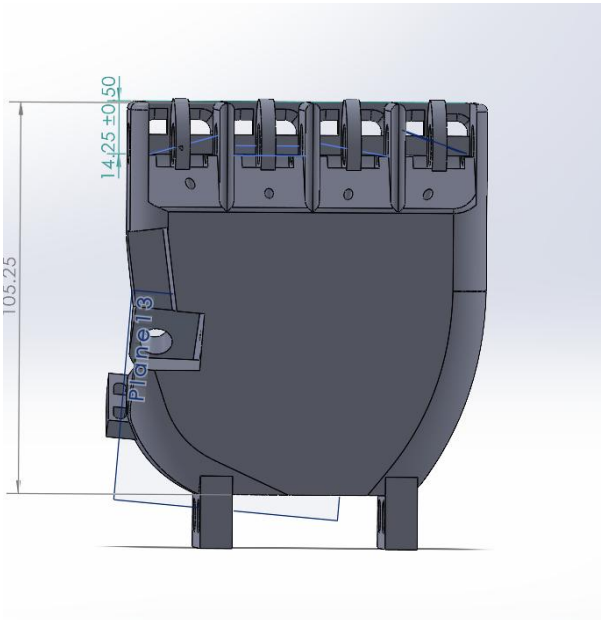


Figure 18: V3 palm concept

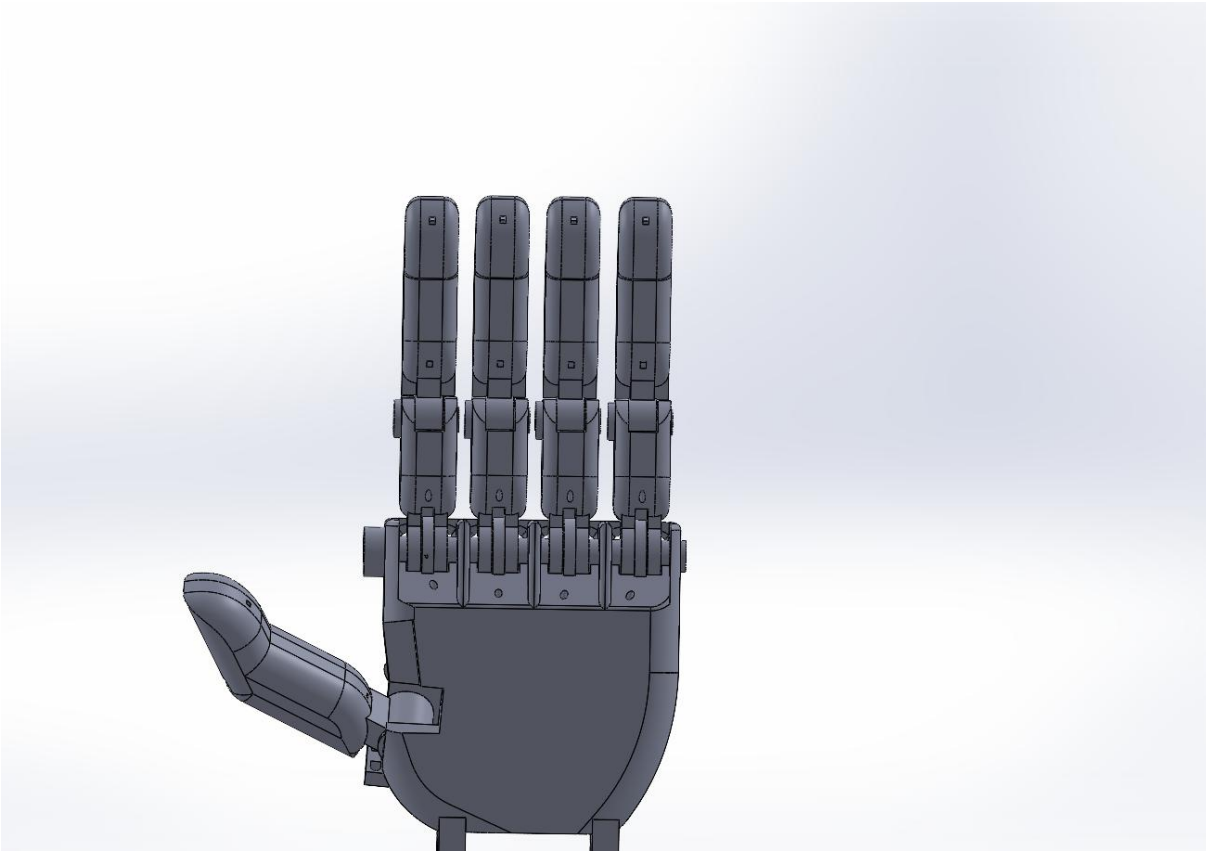


Figure 19: V3 Hand No forearm

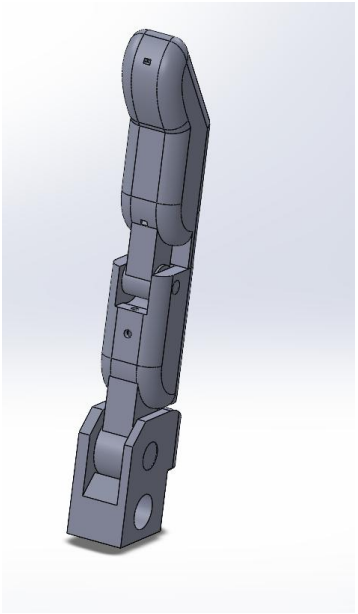


Figure 20: V4 Finger Concept

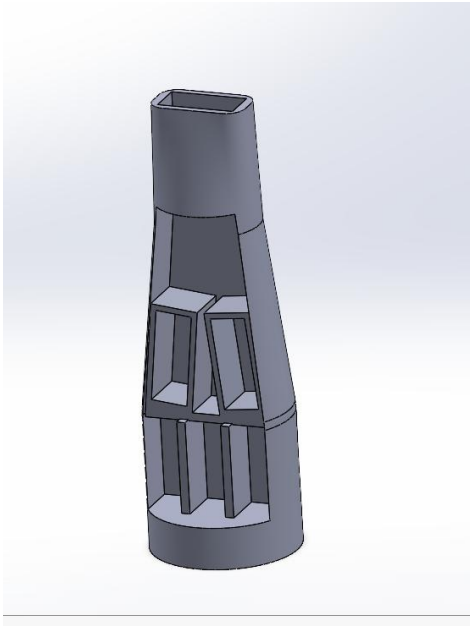


Figure 21: V4 Forearm Concept

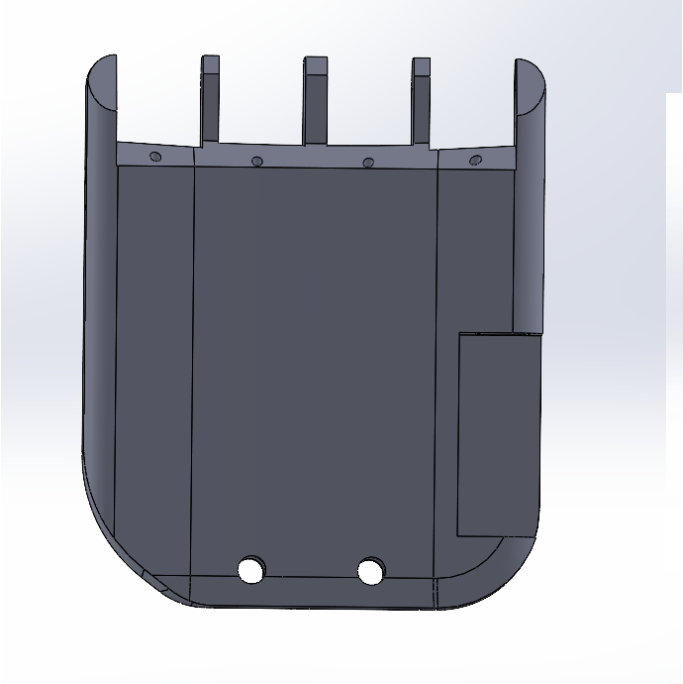


Figure 22: V4 Palm

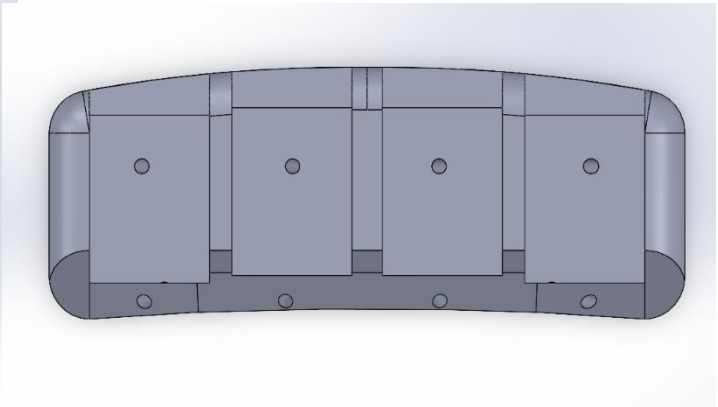


Figure 23: V4 Palm top view

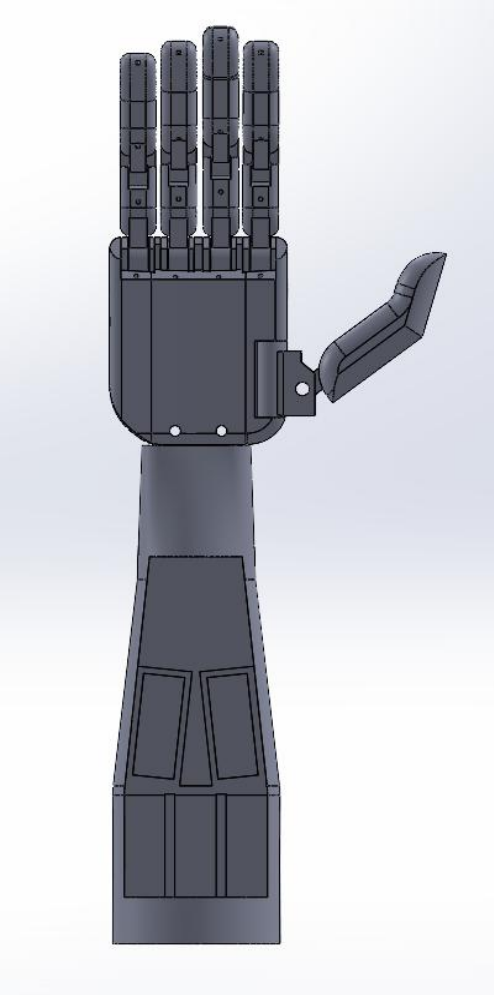


Figure 24: V4 Full model

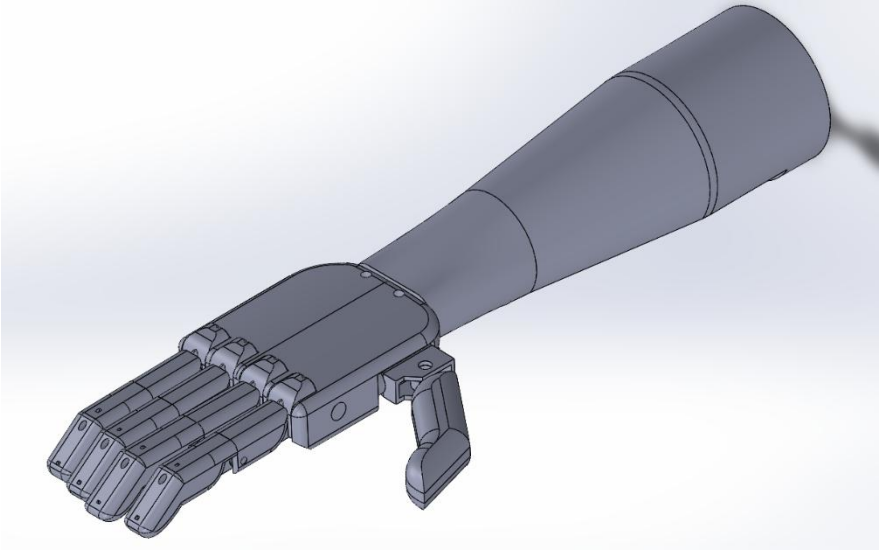
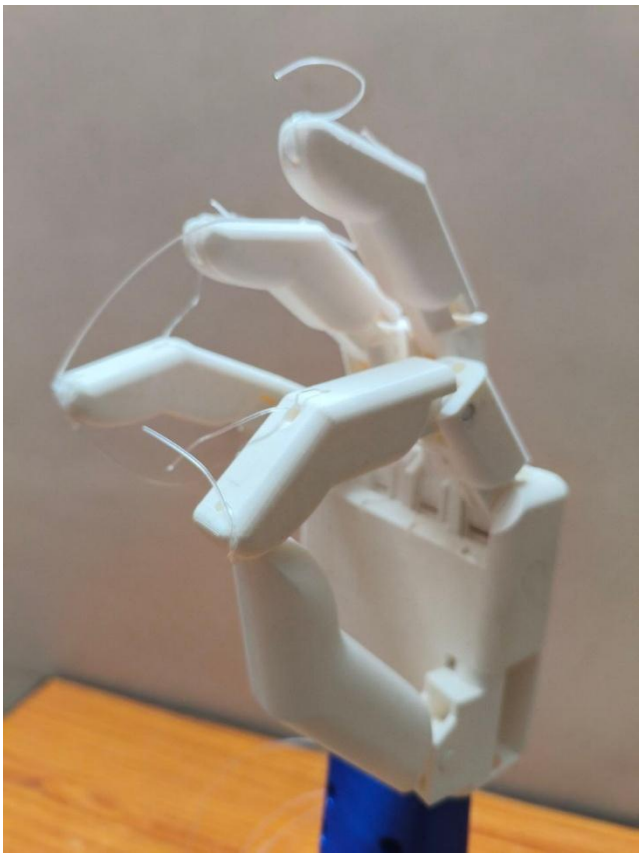


Figure 25: V4 Full hand side view





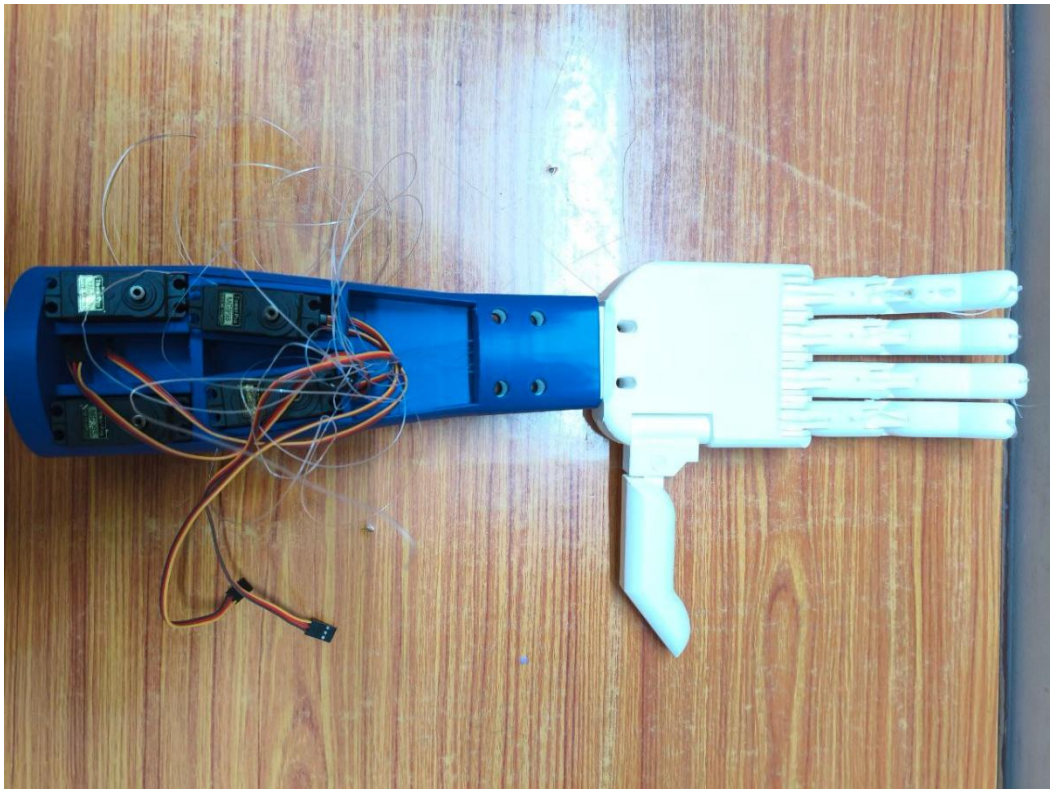


Figure 25: 3D printed version of final model with wiring and Servo-meters